**DBMS-Database Relationship Analysis Using Joins**

**Snapdeal_e-commerce_project**

## Query 1: Combine Customer, Product, Order and Payment Tables

```
SQL> SELECT
  2      C.Customer_ID,
  3      C.Customer_Name,
  4      O.Order_ID,
  5      P.Product_ID,
  6      P.Product_Name,
  7      O.Quantity,
  8      P.Price,
  9      O.Total_Amount,
 10      PY.Payment_ID,
 11      PY.Payment_Method,
 12      PY.Payment_Status
 13  FROM Customer C
 14  INNER JOIN Orders O
 15      ON C.Customer_ID = O.Customer_ID
 16  INNER JOIN Product P
 17      ON O.Product_ID = P.Product_ID
 18  LEFT JOIN Payment PY
 19      ON O.Order_ID = PY.Order_ID;

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
  ORDER_ID PRODUCT_ID
---------- ----------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE TOTAL_AMOUNT PAYMENT_ID PAYMENT_METHOD
---------- ---------- ------------ ---------- ------------------------------
PAYMENT_STATUS
--------------------
       101
```

## Query 2: INNER JOIN

```
SQL> SELECT
  2      C.Customer_ID,
  3      C.Customer_Name,
  4      O.Order_ID,
  5      O.Total_Amount
  6  FROM Customer C
  7  INNER JOIN Orders O
  8      ON C.Customer_ID = O.Customer_ID;

CUSTOMER_ID
-----------
CUSTOMER_NAME
----------------------------------------------------
  ORDER_ID TOTAL_AMOUNT
---------- ------------
       101
Rahul
      1001        50000

       102
Priya
      1002        20000
```

**Query 3: LEFT JOIN**

```
SQL> SELECT
  2      C.Customer_ID,
  3      C.Customer_Name,
  4      O.Order_ID,
  5      O.Order_Status
  6  FROM Customer C
  7  LEFT JOIN Orders O
  8      ON C.Customer_ID = O.Customer_ID;

CUSTOMER_ID
-----------
CUSTOMER_NAME
----------------------------------------------------------
  ORDER_ID ORDER_STATUS
---------- --------------------
       101
Rahul
      1001 Delivered

       102
Priya
      1002 Shipped
```

**Query 4: RIGHT JOIN**

```
SQL> SELECT
  2       C.Customer_ID,
  3       C.Customer_Name,
  4       O.Order_ID,
  5       O.Total_Amount
  6  FROM Customer C
  7  RIGHT JOIN Orders O
  8       ON C.Customer_ID = O.Customer_ID;

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------
  ORDER_ID TOTAL_AMOUNT
---------- ------------
         101
Rahul
      1001        50000

         102
Priya
      1002        20000
```

**Query 5: Retrieve Complete Order Details**

```
SQL> SELECT
  2      O.Order_ID,
  3      C.Customer_Name,
  4      P.Product_Name,
  5      O.Quantity,
  6      P.Price,
  7      O.Order_Date,
  8      O.Total_Amount,
  9      O.Order_Status,
 10      PY.Payment_Method,
 11      PY.Payment_Status,
 12      PY.Amount
 13  FROM Orders O
 14  INNER JOIN Customer C
 15      ON O.Customer_ID = C.Customer_ID
 16  INNER JOIN Product P
 17      ON O.Product_ID = P.Product_ID
 18  LEFT JOIN Payment PY
 19      ON O.Order_ID = PY.Order_ID;

  ORDER_ID
----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
---------- ---------- --------- ------------ --------------------
PAYMENT_METHOD                 PAYMENT_STATUS           AMOUNT
------------------------------ -------------------- ----------
      1001
Rahul
Laptop
```

## Query 6: Display Customer Purchase History

```
SQL> SELECT
  2      C.Customer_ID,
  3      C.Customer_Name,
  4      O.Order_ID,
  5      P.Product_Name,
  6      O.Quantity,
  7      O.Order_Date,
  8      O.Total_Amount
  9  FROM Customer C
 10  INNER JOIN Orders O
 11      ON C.Customer_ID = O.Customer_ID
 12  INNER JOIN Product P
 13      ON O.Product_ID = P.Product_ID;

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
  ORDER_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY ORDER_DAT TOTAL_AMOUNT
---------- --------- ------------
        101
Rahul
      1001

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
  ORDER_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY ORDER_DAT TOTAL_AMOUNT
---------- --------- ------------
Laptop
         1 01-OCT-26        50000
```

## Query 7: Generate Multi-Table Report

```
SQL> SELECT
  2      C.Customer_Name,
  3      O.Order_ID,
  4      P.Product_Name,
  5      O.Total_Amount,
  6      O.Order_Status,
  7      PY.Payment_Method,
  8      PY.Payment_Status
  9  FROM Customer C
 10  INNER JOIN Orders O
 11      ON C.Customer_ID = O.Customer_ID
 12  INNER JOIN Product P
 13      ON O.Product_ID = P.Product_ID
 14  LEFT JOIN Payment PY
 15      ON O.Order_ID = PY.Order_ID;

CUSTOMER_NAME
--------------------------------------------------------------------------------
  ORDER_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_AMOUNT ORDER_STATUS         PAYMENT_METHOD
------------ -------------------- ------------------------------
PAYMENT_STATUS
--------------------
Rahul
      1001
Laptop

CUSTOMER_NAME
--------------------------------------------------------------------------------
  ORDER_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_AMOUNT ORDER_STATUS         PAYMENT_METHOD
------------ -------------------- ------------------------------
PAYMENT_STATUS
--------------------
       50000 Delivered            UPI
Paid
```

## Conclusion:

Thus, database relationships were analyzed using INNER JOIN, LEFT JOIN, and RIGHT JOIN. Customer, Product, Orders, and Payment tables were combined to retrieve complete order details, customer purchase history, and multi-table reports. Hence, SQL JOIN operations help to retrieve related data from multiple tables efficiently.